

NLP HW3 — Q2: WhoDunnit (Multi-Agent Systems)

Gal Barak

July 2026

2.1 Naive Agent Implementation (Analysis)

We ran the naive baseline with the default configuration and no additional flags (`python run_game.py --iterations 3`: 3 suspects, `max_turn_count = 11`, no summarizer, no internal thought). *Alex* is the Accuser and *Beth* the Intel agent.

What specific behavioral problems did you identify in the initial run of the naive agents?

Problem A — Analysis paralysis: the Accuser will not accuse even when a single answer already fixes the culprit. In **Game 1** (culprit = Suspect 3) only Suspect 3 wears square glasses, so Beth's answer `square eye glasses` → `Suspect 3` uniquely determined the culprit on the second question. Alex nonetheless asked four more broad questions and never accused, hitting the turn limit.

Problem B — No cross-turn reasoning: the Accuser never intersects the narrowing suspect lists into a running candidate set. In **Game 2** (culprit = Suspect 3) no single answer pins the culprit; it emerges only from combining `blue eye color` → `S2, S3` with `circular eye glasses` → `S1, S3`, whose intersection is `{S3}`. Because Alex treats each answer in isolation, it never computes this intersection, keeps questioning, and times out. This inability to accumulate evidence across turns is the underlying cause of the analysis paralysis in Problem A.

Do the agents dynamically utilize the full spectrum of their available action space, or do they prematurely converge on a single, repetitive action type?

They **converge**. Alex issues only `request-broad` in all three games (zero `request-specific`; `accuse` only once, in Game 3), which in turn forces Beth into `respond-broad` only. Because the terminal `accuse` action is under-used, information accumulates but is rarely acted upon, so games stall: Games 1 and 2 exhaust all 11 turns without an accusation, ending in timeout rather than a decision.

Does the Accuser make a blind accusation prematurely, or does it get stuck in an endless loop until the max-turn-count is reached? Why do you think it struggles to finalize the decision?

It gets stuck in an **endless questioning loop**, not a premature accusation: Games 1 and 2 reach the `max-turn-count` with no accusation, and Game 3 accuses correctly only after redundant confirmation. It struggles to finalize because:

1. **Stateless prompting** — no maintained candidate set, so it never sees that one suspect remains;
2. the naive prompt **forbids reasoning**, so the model defaults to asking one more question rather than committing;
3. **no firm stopping criterion**, so “accuse when certain” never triggers.

2.2 Context Management via Summarization

We implemented `summarize` in `SummerizerAgent` as an evidence-preserving LLM rewrite of the channel (`temperature=0.2`, prompt instructed to keep every per-suspect confirmed/excluded attribute and never invent facts) and ran with the summarizer enabled. The Summarizer is *Charlie*.

Did the summarizer inadvertently remove any critical clues that led to the Accuser failing to identify the culprit?

No — no clue was dropped. Every decisive fact survived, including the culprit-identifying one (only Suspect 3 has blue eyes). The failure came from two other distortions. First, Charlie downgraded *certainty*: Beth’s definite **circular eye glasses** → **Suspect 1, Suspect 2, Suspect 3** became “Suspect 1: *possibly* has circular eye glasses” (likewise S2, S3), turning a confirmed fact ambiguous. Second, the per-suspect summary replaced the verbatim Q&A record, so Alex lost track of which questions it had already asked. Both pushed Alex to re-ask “circular eye glasses” and “blue eye color” until it timed out — despite the culprit clue being present the whole time.

Did the reduction in context length result in more coherent questions responses from the Accuser and Intel agents, or did the loss of “raw” dialogue history make their behavior less natural?

Mixed. The questioning became **more coherent** in 2 of 3 games: the per-suspect summaries (e.g. Game 2: “Suspect 2: blue eye color, wears square eye glasses, has long hair”) kept Alex focused on one suspect and pushed it toward targeted **request-specific** questions, unlike the naive baseline’s endless broad queries. But coherence did not translate into success — **all three summarizer games still timed out** (0/3, versus 1/3 for the naive baseline). Replacing the raw dialogue with a summary removed Alex’s memory of what it had already asked, so it kept re-confirming known traits (Game 1’s vague “possibly has” made this worse). So compression made the *style* of questioning cleaner but the *behavior* less effective: the raw history, for all its bulk, was what let Alex avoid repeating itself.

Based on your observations, do you believe this summarization approach would scale to a game with a much larger suspect pool (e.g., suspect-count=20)?

Not without changes. (1) A free-text LLM rewrite is lossy: the “possibly” distortion seen at 3 suspects will compound at 20, and a single weakened or dropped fact can leave the culprit ambiguous. (2) The summary must record facts for many suspects, so its length grows with the pool and the “concise channel” benefit erodes. (3) The real bottleneck is the Accuser’s reasoning and stopping ability, which summarization does not address — in fact it worsened matters at $N=3$ (0/3 solved vs. 1/3 naive), since a third of all turns are consumed by summarizing. A deterministic, structured state (a maintained per-suspect table of confirmed/excluded attributes) would scale far better than a natural-language summary.

2.3 Implementing Reasoning Agents

We ran with the `--allow-internal-thought` flag and implemented `take_turn_thought` for both the Intel and the Accuser. Each agent may now reason freely, but still emits the environment’s required final ACTION/INFO block.

Detail your architectural decisions regarding prompt engineering. What specific intermediate reasoning steps did you introduce to guide the agents, and what did you want each agent to analyze internally before generating its environment-compliant response?

Each agent reasons freely in the prompt but must end with the environment’s ACTION/INFO block; we read only the *last* such block, and for an accusation we extract the suspect number and reformat it to the exact "Suspect N" string the environment requires. The reasoning steps we asked each agent to perform internally were:

- **Accuser** — (1) extract the culprit’s attributes from its description; (2) for every suspect, mark which attributes are confirmed/excluded and maintain the set of suspects consistent with *all* evidence so far; (3) accuse iff exactly one candidate remains, otherwise ask the single most-informative, non-repeated question. The intended artifact is the *running candidate intersection* — exactly what the naive agent never computed — and we pass `turns_left` so it trades certainty against the clock.
- **Intel** — (1) classify the question as *specific* (Yes/No) or *broad*; (2) identify the relevant attribute; (3) check each suspect against it, so broad lists are complete and Yes/No answers correct.

Evaluate whether this “Chain-of-Thought” mechanism successfully mitigated the Task 1 failure modes. Did adding this internal reasoning step improve agent coordination, action-space utilization, or overall success rate compared to the naive baseline?

Yes, clearly, on all three axes:

- **Coordination:** Intel’s lists are accurate and the Accuser *consumes* them by intersecting — e.g. Game 1 (`black hat`→{S3}, `square glasses`→{S3}) and Game 3 (`green eyes`→{S1,S2} ∩ `red shoes`→{S1,S3} = {S1}).
- **Action-space utilization:** the Accuser now actually reaches `accuse` and mixes broad with specific questions; the terminal action is no longer neglected. Intel likewise exercised both actions: `respond-broad` throughout the 3-suspect runs and `respond` (Yes/No) in the scaling runs (e.g. `Does Suspect 3 have blue eyes?`→*Yes* at $N=5$, and `Does Suspect 8 have a bronze watch?`→*Yes* at $N=16$).
- **Success rate:** 2/3 solved in the 3-suspect batch, versus 1/3 for the naive baseline. The endless-questioning loop is largely fixed.

If the agents still fail, discuss the underlying causes. If successful, scale the suspect count and determine the maximum your agents can process (subject to the turn limit and API rate restrictions); analyze the factors that contribute to failure at scale.

Residual failure. In one 3-suspect game (Game 2) the intersection was already {S3} after just *two* broad answers (`black hat`→{S1,S3} ∩ `brown eyes`→{S2,S3} = {S3}), yet the Accuser asked three more broad questions and then a redundant specific question, and timed out. This is a symptom of *over-verification* (see below): the model reaches the answer but keeps asking, and with only ≈ 6 Accuser moves at the 11-turn limit, one or two wasted questions cause a timeout.

Scaling. We increased the suspect count with the turn limit set to $3\times$ suspects, and the agents solved every configuration correctly from 5 up to **32 suspects**, always identifying and accusing the culprit. $N=32$ is the largest we could test: an $N=64$ game needs ≈ 192 turns, each re-embedding all 64 descriptions, which exceeds the Groq free-tier daily token cap (Limit 100000) outright, so those attempts never completed. The cap therefore bounds *throughput*, not the agents’ reasoning, and $N=32$ stands as the demonstrated maximum under the free tier.

Factors that limit scaling. (1) **Over-verification** is the dominant inefficiency: the Accuser consistently asks several more questions than needed to reach a singleton (e.g. 5 needed vs. 11 asked at $N=32$). This is harmless when turns are plentiful but is precisely what causes timeouts under a tight limit — so the required turn budget, not correctness, is the practical ceiling. (2) **API budget:** each 70B turn (≈ 1024 -token thoughts plus an Intel prompt that grows with N) is token-expensive, so throughput — how many and how large the games can be per day — is bounded by the free-tier daily cap even when a single game’s reasoning would succeed. (3) **Intel accuracy at scale:** with 20+ descriptions to scan per broad question there is more room for a missed or misclassified suspect, which would corrupt the intersection.

Propose three modifications that could further enhance agent performance and improve the overall success rate of the system.

1. **Deterministic candidate tracking in code.** Maintain a per-suspect table of confirmed/excluded attributes parsed from the Q&A and auto-accuse once a single candidate remains; let the LLM only *choose questions*. This removes reliance on the model’s mental set-intersection and directly eliminates the over-verification that causes timeouts.
2. **Information-maximizing, de-duplicated questions.** Instruct the Accuser to pick the attribute whose answer most evenly splits the remaining candidates (binary-search/entropy style) and forbid repeating any prior question — fewer turns needed, so larger pools fit under the turn limit.
3. **Token-budget efficiency.** Cap reasoning length, use the small model for the narrowing questions and the 70B model only for the final accusation, and add a `time.sleep/backoff-and-retry` on 429 — so larger suspect pools fit within the daily and per-minute limits.